week3 예습과제

다중 선택	week3

분류

1. 분류의 개요

분류 : 기존 데이터가 어떤 레이블에 속하는지 패턴을 알고리즘으로 인지한 뒤에 새롭게 관측된 데이터에 대한 레이블을 판별

- 분류 알고리즘
  - 나이브 베이즈(Navie Bayes) : 베이즈 통계와 생성 모델에 기반
  - 로지스틱 회귀(Logistic Regression) : 독립변수와 종속변수의 선형 관계성에 기반
  - 결정 트리(Deicison Tree) : 데이터 균일도에 따른 규칙 기반
    - 앙상블의 기본 알고리즘으로 일반적으로 사용됨
      - 과적합으로 인한 성능 저하 단점 → 앙상블 기법에서는 장점
        ⇒ 앙상블은 매우 많은 여러 약한 학습기를 결합해 확률적 보완과 오류 발생 부분에 대한 가중치 업데이트로 예측 성능 향
  - 서포트 벡터 머신(Support Vector Machine) : 개별 클래스 간의 최대 분류 마진을 효과적으로 찾아줌
  - 최소 근접(Neareset Neighbor) 알고리즘 : 근접 거리 기준
  - 신경망(Neural Network) : 심층 연결 기반
  - **앙상블**(Ensemble) : 서로 다른(또는 같은) 머신러닝 알고리즘을 결합
    - 배깅(Bagging)
      - 랜덤포레스트(Random Forest)
    - 부스팅(Boasting)
      - 그래디언트 부스팅(Gradient Boasting) : 뛰어난 예측 성능 but 수행 시간 ↑

→ 최적화 모델 튜닝 어렵

- XgBoost와 LightGBM : 기존 그래디언트 부스팅의 예측 성능 발전 + 수행 시간 단축

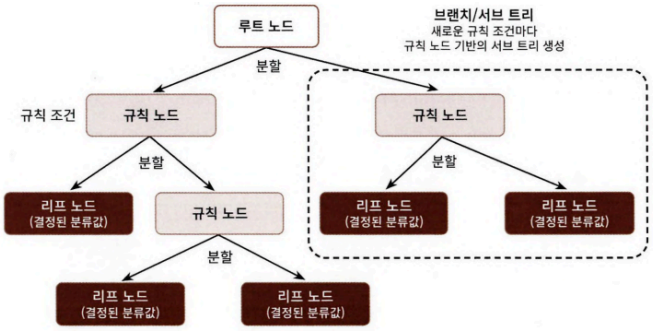
---

2. 결정 트리

: if/else를 자동으로 찾아내 예측을 위한 규칙을 만드는 알고리즘
→ 어떤 기준을 바탕으로 규칙을 만들지가 성능 좌우

0) 결정트리

(1) 간단한 결정 트리 구조



규칙 노드(=결정 노드, 브랜치 노드) : 규칙 조건

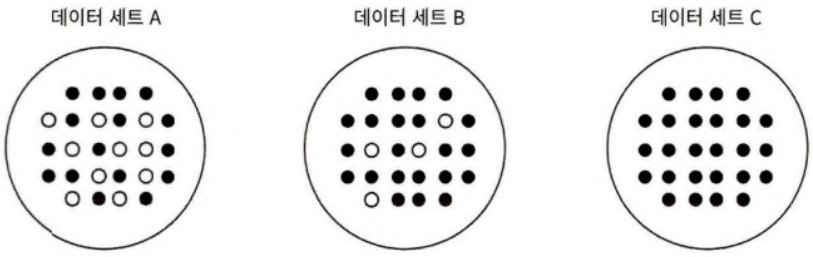
리프 노드(= 말단 노) : 결정된 클래스 값

서브 트리 : 새로운 규칙 조건마다 생성됨

- 규칙 ↑ → 분류 결정 방식 복잡 → 과적합 가능성 ↑
- 트리의 깊이 ↑→ 결정 트리의 예측 성능 저하 가능성 ↑

⇒ 최대한 균일한 데이터 세트로 구성하도록 분할하는 것이 중요

(2) 정보 균일도



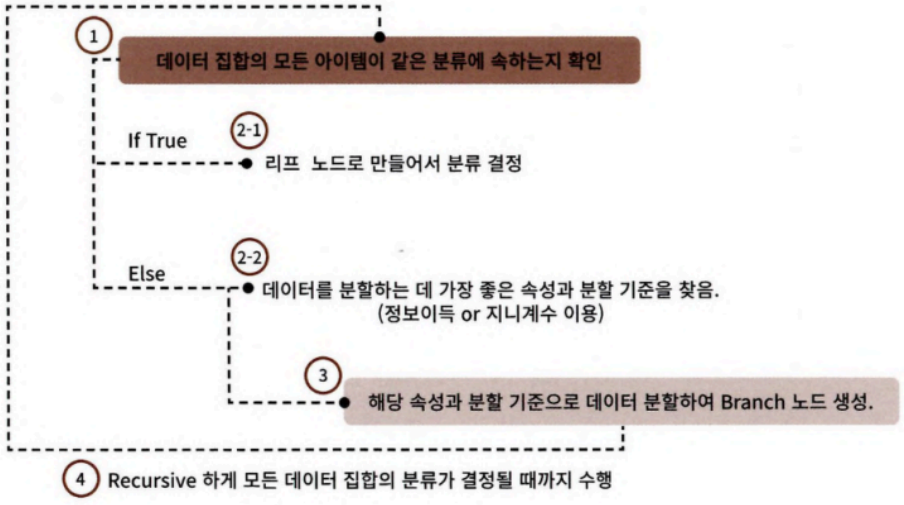
균일도 : C > B > A

데이터 세트 C에서는 별다른 정보 없이도 '검은 공'이라고 쉽게 예측 가능
but 데이터 세트 A에서는 혼잡도↑+균일도↓ → 같은 조건에서 데이터 판단에 더 많은 정보 필요

⇒ 결정 노드는 정보 균일도가 높은 데이터 세트를 먼저 선택하도록 규칙 조건을 만듦
→ 균일도가 높은 데이터 세프 먼저 골라내고 남은 데이터에서 균일도가 높은 데이터를 다시 골라내기를 반복

(3) 정보 균일도 측정 방식

- 정보 이득(Information Gain) 지수
  - 엔트로피(주어진 데이터 집합의 혼잡도) 개념 기반
  - 1 - 엔트로피 지수
  - 결정 트리는 정보 이득 지수로 분할 기준을 정함 → 정보 이득이 높은 속성 기준으로 분할
- 지니 계수
  - 원래 경제학에서 불평등 지수를 나타낼 때 사용하는 계수
  - 0이 가장 평등, 1이 가장 불평등
  - 지니 계수가 낮을수록 데이터 균일도가 높음 → 지니 계수가 낮은 속성 기준으로 분할
  - DecisonTreeClassifier에서 사용



1) 결정 트리 모델의 특징

- 장점

    - '균일도'라는 룰 기반
        - 알고리즘이 쉽고 직관적
            - 룰이 명확, 어떻게 규칙 노드와 리프 노드가 만들어지는지 파악 가능, 시각화 가능
        - 정보의 균일도만 신경 쓰면 됨
            - 피처의 스케일링이나 정규화 등의 사전 가공 영향도가 적음
- 단점
    - 과적합으로 정확도가 떨어짐
        - 피처가 많고 균일도가 다양하게 존재할수록 트리 깊이↑+복잡
        - 완벽한 규칙을 만들지 못하는 경우에도 정확도를 높이기 위해 조건 추가 → 복잡한 모델 → 실제 상황에 유연한 대처 불가 → 예측 성능 저하

2) 결정 트리 파라미터

(1) 사이킷런의 결정 트리 알고리즘 클래스

- DecisionTreeClassifier
- DecisionTreeRegressor
- 둘 다 CART 알고리즘 기반

(2) DecisonTreeClassifier

파라미터 종류

파라미터 명	설명
min_samples_split	• 노드 분할을 위한 최소한의 샘플 데이터 수 • 과적합 제어 • 디폴트 20 / 작게 설정할수록 분할 노드수 ↑ → 과적합 가능 ↑
min_samples_leaf	• 분할이 될 경우 왼/오른쪽의 브랜치 노드에서 가져야 할 최소한의 샘플 데이터 수 • 큰 값으로 설정될수록 조건 만족이 어려움 → 노드 분할 상대적으로 덜 수행 • 과적합 제어 용도 • but 비대칭적(imbalanced) 데이터의 경우 특정 클래스의 데이터가 극도로 작을 수 있어 이 경우 작게 설정 필요
max_features	• 최적의 분할을 위해 고려할 최대 피처 개수 • 디폴트 None → 모든 피처를 사용 • int 형으로 지정하면 대상 피처 개수 / float 형으로 지정하면 전체 피처에서

	의 대상 피처의 비율 • ‘sqrt’ : 전체 피처 중 sqrt(전체 피처 개수)만큼 선정 • ‘auto’ : sqrt와 동일 • ‘log’ : 전체 피처 중 log2(전체 피처 개수) 선정 • ‘None’ : 전체 피처 선정
max_depth	• 트리의 최대 깊이 규정 • 디폴트 None → 완벽하게 클래스 결정 값이 될 때까지 깊이 증가 or 노드가 가지는 데이터 개수가 min_samples_split보다 작아질 때까지 계속 깊이 증가 • 깊이 ↑이면 min_samples_split 설정대로 최대 분할 → 과적합할 수 있음 → 적절한 값으로 제어 필요
max_leaf_nodes	• 말단 노드의 최대 개수

3) 결정 트리 모델의 시각화

Graphviz 패키지 이용

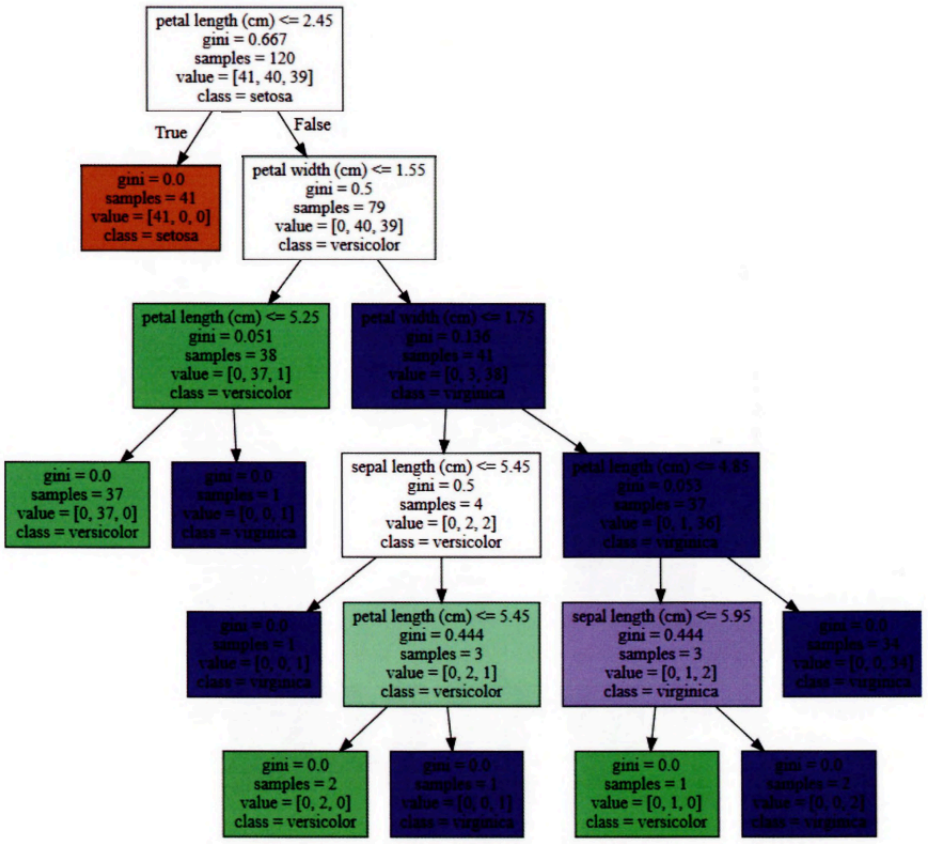
그래프 기반의 dot 파일로 기술된 다양한 이미지를 쉽게 시각화 가능

사이킷의 export_graphviz( ) API로 인터페이스 가능

- 함수 인자
  - estimator : 학습이 완료된 객체
  - out_file : 출력 파일명
  - feature_names : 피처 이름 리스트
  - class_names : 클래스 이름 리스트

→ 실제 트리 형태로 시각화

예시와 함께 보기



리프(말단) 노드	• 더 이상 자식 노드가 없는 노드 • 최종 클래스(레이블) 값이 결정되는 노드 • 오직 하나의 클래스 값으로 최종 데이터가 구성 or 리프노드가 될 수 있는 하이퍼 파라미터 조건 충족
브랜치 노드	자식노드가 있는 노드, 자식 노드를 만들기 위한 분할 규칙 조건이 있음

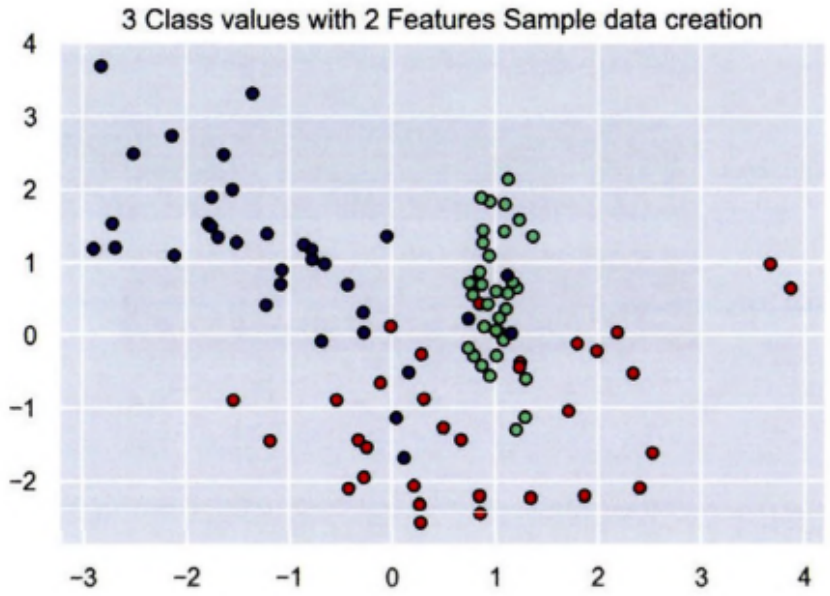
gini	value=[]로 주어진 데이터 분포에서의 지니 계수
samples	현 규칙에 해당하는 데이터 건수
value = []	클래스 값 기반의 데이터 건수 레이블 수에 따라 차례대로 구성 됨
맨 위 부등식	분할 조건
class=' '	하위노드 가질 경우 가장 많은 레이블
노드 색	데이터의 레이블 값 색이 짙어질수록 지니 계수↓ + 해당 레이블에 속하는 램플 데이터 ↑

결정 트리는 균일도에 기반해 어떠한 속성을 규칙 조건으로 선택하느냐가 중요한 요건
→ 중요한 몇개의 피처가 명확한 규칙 트리를 만드는 데 크게 기여 → 모델을 간결+이상치에 강한 모델 만들 수 있음

- DecisionTreeClassifier객체의 feature_importances_ 속성
  - ndarray 형태로 반환
  - 피처 순서대로 값 할당 ex) [0.015 0.025 0960]
  - 피처가 트리 분할 시 정보 이득이나 지니 계수를 얼마나 효율적으로 잘 개선 시켰는지를 정규화된 값으로 표현
  - 일반적으로 값이 높을수록 해당 피처의 중요도가 높다는 의

4) 결정 트리 과적합(Overfitting)

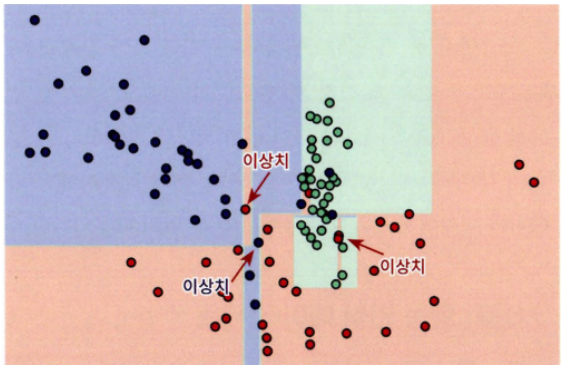
(1) 데이터 분포 산점도

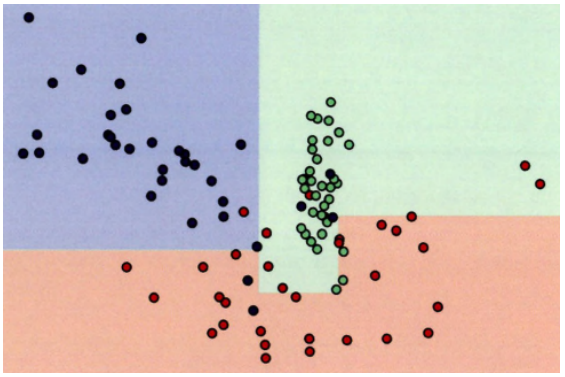


(2) 결정 경계 시각화

하이퍼 파라미터를 디폴트로 했을 때
→ 일부 이상치 데이터까지 분류하기 위 경계↑
→ 예측 정확도 ↓

min_samples_leaf=6으로 트리 생성 조건을 제약했을 때
→ 이상치에 크게 반응하지 않고 좀 더 일반화





5) 결정 트리 실습 - 사용자 행동 인식 데이터 세트

---

3. 앙상블 학습

1) 앙상블 학습 개요

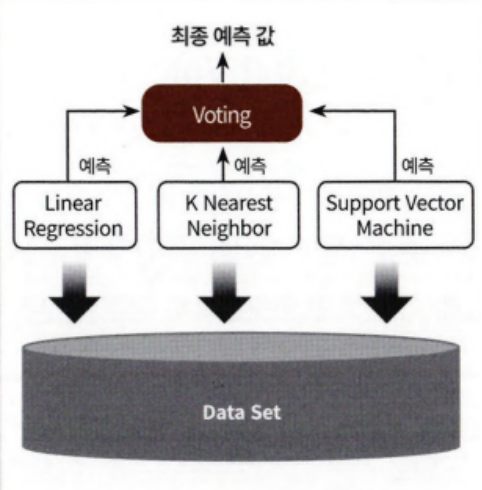
(1) 앙상블 학습

: 여러 개의 분류기를 생성하고 그 예측을 결합함으로써 보다 정확한 최종 예측을 도출하는 기법

- 대부분의 정형 데이터 분류 시에는 앙상블이 딥러닝보다 뛰어난 성능을 나타냄

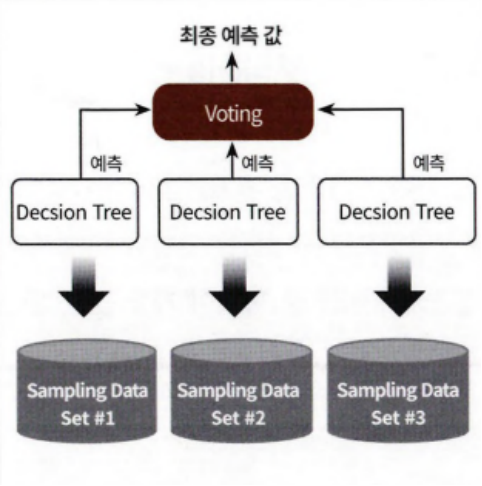
(2) 보팅

- 여러 개의 분류기가 투표를 통해 최종 예측 결과를 결정
- 서로 다른 알고리즘을 가진 분류기를 결합



(3) 배깅

- 여러 개의 분류기가 투표를 통해 최종 예측 결과를 결정
- 각각의 분류기가 모두 같은 유형의 알고리즘 기반이지만 데이터 샘플링을 서로 다르게 가져가면서 학습을 수행
- 부트스트래핑 분할 방식 : 개별 분류기에 할당된 학습 데이터는 원본 학습 데이터를 샘플링해 추출 (중첩 허용)



(4) 부스팅

- 여러 개의 분류기가 순차적으로 학습 수행
- 앞에서 학습한 분류기가 예측이 틀린 데이터에 대해서는 올바르게 예측할 수 있도록 다음 분류기에게 가중치를 부여하면서 학습과 예측을 진행
- 부스팅 방식 : 가중치를 부스팅하면서 학습을 진행
- 종류

- 그래디언트 부스팅
- XGBoost
- LightGBM

(5) 스태킹

: 여러 가지 다른 모델의 예측 결괏값을 다시 학습 데이터로 만들어서 다른 모델(메타 모델)로 재학습시켜 결과를 예측하는 방법

2) 보팅 유형 - 하드 보팅(Hard Voting)과 소프트 보팅(Soft Voting)

(1) 하드 보팅

- 다수결 원칙
- 예측한 결괏값들 중 다수의 분류기가 결정한 예측값 = 최종 보팅 결괏값

(2) 소프트 보팅

- 분류기들의 레이블 값 결정 확률을 모두 더하고 이를 평균해서 이들 중 확률이 가장 높은 레이블 값 = 최종 보팅 결괏값
- 일반적으로 소프트 보팅이 보팅 방법으로 적용 됨

3) 보팅 분류기(Voting Classifier)

- 사이킷런은 보팅 방식의 앙상블을 구현한 VotingClassifier 클래스 제공

- VotingClassifier 클래스 주요 생성 인자
  - estimators : 리스트 값으로 보팅에 사용될 여러 개의 Classifier 객체들을 튜플 형식으로 입력
  - voting : ‘hard’(기본 값) 또는 ‘soft’

- ML 모델의 성능은 다양한 테스트 데이터에 의해 검증 됨 → 높은 유연성을 가지는게 중요
- 배깅&부스팅은 대부분 결정 트리 알고리즘 기반
  → 결정 트리 알고리즘의 단점을 수십~수천 개의 분류기를 결합

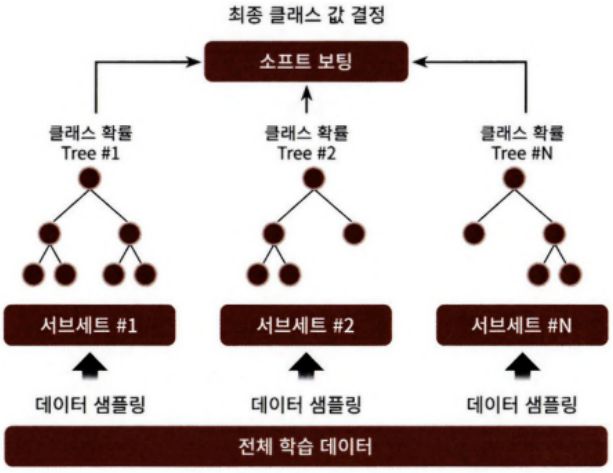
→ 결정 트리 알고리즘의 장점은 그대로 취하고 단점은 보완하여 편향-분산 트레이드오프 효과 극대화 가능

---

4. 랜덤 포레스트

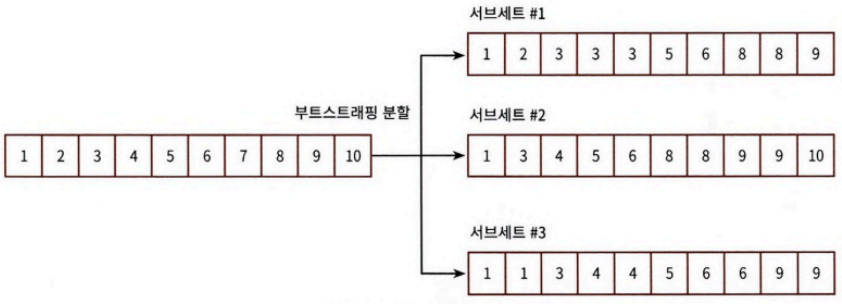
1) 랜덤 포레스트의 개요

- 특징
  - 배깅의 대표적인 알고리즘
  - 앙상블 알고리즘 중 비교적 빠른 수행 속도
  - 다양한 영역에서 높은 예측 성능을 보임
  - 기반 알고리즘 : 결정 트리 → 직관적
- 과정 :
  여러 개의 결정 트리 분류기가 전체 데이터에서 배깅 방식으로 각자의 데이터를 샘플링해 개별적으로 학습을 수행 → 최종적으로 모든 분류기가 보팅을 통해 예측을 결정



〈 랜덤 포레스트 〉

- 부트스트래핑(bootstrapping) 분할 방식
  - 여러 개의 데이터 세트를 중첩되게 분리
  - 서브세트의 데이터 건수는 전체 데이터 건수는 동일하지만 개별 데이터가 중첩되어 만들어짐
  - ex)



〈 부트스트래핑 샘플링 방식 〉

- 사이킷런의 RandomForestClassifier 클래스 이용

2) 랜덤 포레스트 하이퍼 파라미터 및 튜닝

- 파라미터 종류

n_estimators	• 결정 트리 개수 • 디폴트 10개 • 많이 설정할수록 좋은 성능 기대 but 많이할수록 꼭 성능이 향상되는 건 X + 학습 수행 시간 증가
max_features	• 기본 값 'auto'(='sqrt') • 트리 분할시 전체 피처가 아니라 sqrt(전체 피처)만큼 참조
max_depth	• 과적합 개선
min_samples_leaf	• 과적합 개선
min_samples_split	• 과적합 개선